



IILM UNIVERSITY

School of Computer Science and Engineering

INTERNSHIP REPORT

Python Developer Internship

Internship Projects:

1. AI-Based Sentiment Analyzer
2. Secure Authentication System

Submitted in partial fulfilment of the requirement for the degree of

B.Tech in Computer Science and Engineering

Submitted By

ARNAV RAJ PANDEY

Roll / Enrollment No.: 25SCS1003002322

Batch: 2026-27

Organization: Codec Technologies Pvt. Ltd.

Internship Period: 27/06/2026 – 27/07/2026

1. CANDIDATE'S DECLARATION

I, ARNAV RAJ PANDEY, hereby declare that the internship report titled “Python Developer Internship” has been prepared by me as part of the requirements for the Bachelor of Technology in Computer Science and Engineering at IILM University.

The report describes the training and project work completed during my internship with Codec Technologies Pvt. Ltd. The project descriptions and implementation details are based on the project source files supplied for this internship report.

I have presented the work based on the internship activities and source files available to me. I understand that I am responsible for the accuracy and originality of the material submitted in this report.

Signature: _____

Student Name: ARNAV RAJ PANDEY

Roll / Enrollment No.: 25SCS1003002322

2. ACKNOWLEDGEMENT

I would like to express my sincere gratitude to IILM University and the School of Computer Science and Engineering for providing me with the opportunity to undertake an internship and gain practical exposure to Python development.

I am thankful to Codec Technologies Pvt. Ltd. for providing the internship opportunity and a platform to work on Python-based applications. The internship enabled me to strengthen my understanding of Python fundamentals, libraries, web application development, natural language processing concepts, database usage and authentication mechanisms.

I am also grateful to the internship/training team for their guidance during the internship. The practical work helped me understand how programming concepts are combined to build functional web applications.

Finally, I would like to thank my teachers, family and everyone who supported me during the completion of this internship.

Signature: _____

Student Name: ARNAV RAJ PANDEY

Roll / Enrollment No.: 25SCS1003002322

3. INTERNSHIP COMPLETION CERTIFICATE

The following is the single official Internship Completion Certificate included with this report. No additional certificate is attached.



4. PROJECT DESCRIPTION

4.1 Introduction

The internship was undertaken in the role of Python Developer Intern at Codec Technologies Pvt. Ltd. from 27 June 2026 to 27 July 2026. The work began with Python fundamentals and exposure to Python libraries, followed by practical development work.

Two projects were completed during the internship: an AI-Based Sentiment Analyzer and a Secure Authentication System. Together, the projects provided practical exposure to Python programming, Flask web development, database integration, machine-learning/NLP model usage, password hashing and token-based authentication.

4.2 Organization Profile

Codec Technologies Pvt. Ltd. provided the internship environment in which the Python development work was completed. The internship certificate identifies the organization and the designation of Python Developer Intern.

The supplied project source files show the use of Python, Flask, SQLite, Hugging Face Transformers, PyTorch, bcrypt and PyJWT across the two applications.

4.3 Problem Statement

Project 1 – AI-Based Sentiment Analyzer: Manual interpretation of sentiment expressed in large numbers of text messages, reviews or comments can be time-consuming. The project addresses this by providing a web interface where a user enters text and a pretrained NLP model analyzes it and returns a sentiment presentation with a confidence score.

Project 2 – Secure Authentication System: Web applications need a mechanism for user registration and login while avoiding storage of passwords in plain text. The project implements user registration, password hashing, login verification, session handling and JWT-based token generation, followed by access to a dashboard.

4.4 Project Objectives

- To strengthen practical knowledge of Python programming and libraries.
- To build web-based applications using Flask.
- To apply a pretrained Transformer model for text sentiment analysis.
- To store sentiment analysis history using SQLite.
- To implement user registration and login functionality.
- To hash passwords using bcrypt before storing them.
- To demonstrate JWT token generation and session-based access to a dashboard.
- To gain practical experience in integrating Python, web interfaces and databases.

4.5 Scope of the Project

The sentiment analyzer is designed for short text inputs such as reviews, comments and messages. The submitted implementation uses the `distilbert-base-uncased-finetuned-sst-2-english` pretrained model and stores recent analysis results in SQLite.

The authentication project covers registration, login, dashboard access and logout. Both projects are educational and demonstrate practical Python web-development concepts; they should not be treated as production-ready security systems without further hardening.

4.6 Technologies and Tools Used

Project	Technology / Library	Purpose
Both	Python	Core programming language
Both	Flask	Web application framework
Sentiment Analyzer	Hugging Face Transformers	Pretrained Transformer sentiment pipeline
Sentiment Analyzer	DistilBERT SST-2	Pretrained sentiment model
Sentiment Analyzer	PyTorch	Backend dependency
Sentiment Analyzer	SQLite	Stores sentiment history

Authentication	bcrypt	Password hashing and verification
Authentication	PyJWT	JWT token generation
Authentication	SQLite	Stores user account records
Both	HTML/CSS	Web interface and styling

4.7 System Architecture

Project 1 – AI-Based Sentiment Analyzer

User → Flask Web Interface → Text Validation → Hugging Face Sentiment Pipeline → Sentiment/Confidence Result → SQLite History Storage → Result&Recent History displayed in Browser

The application loads the pretrained model when the Flask application starts. For a submitted text, the implementation sends at most the first 512 characters to the sentiment model. The model returns POSITIVE or NEGATIVE with a score. The application presents Positive or Negative when confidence is at least 85%; otherwise it presents Neutral.

Project 2 – Secure Authentication System

User → Registration/Login Form → Flask Route → SQLite User Record → bcrypt Password Verification → JWT Token Generation → Flask Session → Protected Dashboard → Logout / Session Clear

The authentication application stores a username, email and hashed password in SQLite. During login, the entered password is checked against the stored bcrypt hash. On successful authentication, a JWT containing the user ID and one-hour expiration is generated and stored in the Flask session.

4.8 Methodology

The internship work followed a practical learning approach. The initial phase focused on Python fundamentals and familiarity with libraries. The next phase involved applying those concepts through two web-based projects.

- Requirement understanding and selection of project functionality.
- Creation of the Flask applications and web interfaces.
- Integration of Python libraries and supporting dependencies.
- Implementation of database operations using SQLite.
- Testing the main user flows such as text analysis, registration, login and logout.
- Review of project structure and documentation.

5. PROJECT 1 – AI-BASED SENTIMENT ANALYZER

5.1 Overview

The AI-Based Sentiment Analyzer is a Flask web application that accepts user-entered text and analyzes its sentiment using a pretrained Transformer model. The application uses the Hugging Face Transformers pipeline with `distilbert-base-uncased-finetuned-sst-2-english`.

5.2 Main Features

- Text input through a web form
- Sentiment analysis using a pretrained model
- Positive/Negative/Neutral presentation
- Confidence score display
- Recent analysis history
- SQLite database storage
- Responsive HTML/CSS interface

5.3 Implementation Details

The Flask application defines the main route at `/`. A POST request reads the submitted text and validates that it is not empty. If text is supplied, the application sends up to 512 characters to the sentiment pipeline. The returned model score is converted to a percentage.

The application maps high-confidence POSITIVE/NEGATIVE predictions to Positive/Negative and lower-confidence predictions to Neutral for presentation. Each result is inserted into the SQLite 'analyses' table with text, sentiment, confidence and timestamp.

5.4 Project Structure

- `app.py`
- `requirements.txt`
- `README.md`
- `templates/index.html`
- `static/style.css`
- SQLite database created by the application at runtime

5.5 Outcome

The submitted implementation demonstrates how a pretrained NLP model can be integrated into a Python Flask application and combined with database logging and a browser-based interface.

5.6 Working Flow

- User opens the Flask web application in a browser.
- User enters a review, comment or message into the text area.
- The application validates that the input is not empty.
- The pretrained DistilBERT SST-2 pipeline predicts POSITIVE or NEGATIVE and returns a confidence score.
- The application converts the score to a percentage and applies the 85% presentation threshold.
- The result is displayed as Positive, Negative or Neutral along with confidence.
- The analysis is stored in SQLite and the latest history is displayed.

5.7 Example

Example input: “This product is excellent and I really love it!”

Expected presentation: Positive, together with the model confidence percentage.
The project README provides this example.

5.8 Limitations

The underlying SST-2 model is binary and returns POSITIVE or NEGATIVE. The Neutral category is a presentation rule based on confidence below 85%, not a native third class of the model.

The application limits the text passed to the model to the first 512 characters. Further improvements could include richer validation, more detailed error handling and broader evaluation on domain-specific text.

6. PROJECT 2 – SECURE AUTHENTICATION SYSTEM

6.1 Overview

The Secure Authentication System is a Flask-based web application that provides user registration, login, dashboard access and logout. It uses SQLite for user records, bcrypt for password hashing and PyJWT for generating JWT tokens.

6.2 Main Features

- User registration with username, email and password.
- Unique email constraint in the SQLite database.
- Password hashing with bcrypt.
- Login verification using bcrypt.checkpw.
- JWT token generation with a one-hour expiration.
- Session-based dashboard access.
- Logout through session clearing.

6.3 Database Design

The submitted application creates a 'users' table with the following fields: id (primary key), username, email (unique) and password (stored as a bcrypt hash).

6.4 Authentication Flow

During registration, the password is converted to bytes and hashed using `bcrypt.gensalt()`. The resulting hash is stored in SQLite. During login, the application retrieves the user record by email and checks the entered password against the stored hash.

On successful authentication, a JWT containing the user ID and a one-hour expiration is created using HS256. The token and username are stored in the Flask session. The dashboard checks for the session token before allowing access. Logout clears the session and redirects the user to the login page.

6.5 Application Pages

- Registration Page – collects username, email and password and creates a user record.
- Login Page – accepts email and password and verifies the credentials against the stored bcrypt hash.
- Dashboard – displays a welcome message for the authenticated user and confirms successful login.
- Logout – clears the Flask session and returns the user to the login page.

6.6 Project Structure

- app.py
- requirements.txt
- database.db
- Static.html/style.css
- Templates/register.html
- Templates/login.html
- Templates/dashboard.html

6.7 Outcome

The project demonstrates the practical integration of authentication concepts, password hashing, JWT token creation, sessions, SQLite and Flask routes in a small web application.

6.8 Security Considerations

The submitted code uses hard-coded values for `app.secret_key` and `JWT_SECRET`. For production use, secrets should be kept outside source code and stronger security controls should be applied. The project is best understood as an educational demonstration rather than a production-ready authentication system.

7. LEARNING OUTCOMES

- Improved understanding of Python syntax and programming fundamentals.
- Practical use of Python libraries and third-party packages.
- Experience developing Flask web applications.
- Understanding of HTML/CSS integration with Flask templates.
- Exposure to NLP and pretrained Transformer models.
- Understanding of database operations using SQLite.
- Practical experience with password hashing using bcrypt.
- Understanding of JWT generation and session-based access control.
- Improved ability to organize, test and document Python projects.

8. CONCLUSION

The one-month Python Developer Internship provided practical exposure to Python programming and web application development. After learning Python fundamentals and libraries, two projects were developed: an AI-Based Sentiment Analyzer and a Secure Authentication System.

The first project demonstrated NLP model integration, sentiment classification, confidence display and SQLite history storage. The second demonstrated user registration, bcrypt password hashing, JWT token generation, sessions and dashboard access.

Overall, the internship helped connect Python concepts with practical application development and strengthened my understanding of web development, databases, NLP integration, authentication and systematic problem solving.

9. BIBLIOGRAPHY / REFERENCES

Codec Technologies Pvt. Ltd., Internship Completion Certificate, Python Developer Intern, internship period 27/06/2026–27/07/2026.

Codec Technologies Pvt. Ltd., Python Development Training Program documentation supplied for the internship.

Python Software Foundation, Python Documentation and Standard Library documentation.

Flask Documentation, web application framework reference.

Hugging Face Transformers Documentation, pretrained Transformer models and sentiment-analysis pipeline.

SQLite Documentation, database and SQL reference.

bcrypt package documentation, password hashing and verification.

PyJWT documentation, JSON Web Token creation and encoding.

Submitted project source files: AI-Based Sentiment Analyzer and Secure Authentication System.